// Security Assessment

02.03.2025 - 02.13.2025

Core Strategy Re-Audit

Canopy

HALBORN

Core Strategy Re-Audit - Canopy

Prepared by: HALBORN

Last Updated 03/11/2025

Date of Engagement by: February 3rd, 2025 - February 13th, 2025

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
11	0	0	0	3	8

TABLE OF CONTENTS

1. Introduction

Canopy engaged **Halborn** to conduct a security reassessment of the Canopy project, beginning on February 3rd, 2025, and ending on February 13th, 2025. This security assessment focused on the smart contracts within the satay-movement GitHub repository, commit hashes, and further details can be found in the Scope section of this report.

Canopy is a yield aggregator on the Movement network that enables efficient interaction with multiple DeFi protocols through modular strategies composed of predefined operations. It also provides automated vaults that optimize fund allocations and rebalance strategy debt ratios, offering a streamlined approach to yield generation with reduced user intervention.

2. Assessment Summary

The team at Halborn assigned a full-time security engineer to verify the security of the modules. The security engineer is a blockchain and smart-contract security expert with advanced penetration testing, smart-contract hacking, and deep knowledge of multiple blockchain protocols.

The purpose of this assessment is to:

- Ensure that modules functions operate as intended.
- Identify potential security issues with the modules.

In summary, Halborn identified some improvements to reduce the likelihood and impact of risks, which were successfully addressed by the Canopy team. The main ones were the following:

- Fix the name and symbol assignments in the share metadata.
- Enforce the execution of apply_strategy_loss directly within the Vault module when necessary.
- Assign a distinct error code to each issue.
- Remove the recipient parameter and manually send it as the signer's address from the withdraw function within the LayerBank block.

3. Test Approach And Methodology

Halborn performed a combination of the manual view of the code and automated security testing to balance efficiency, timeliness, practicality, and accuracy regarding the scope of the smart contract assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation, automated testing techniques help enhance the coverage of modules. They can quickly identify items that do not follow security best practices. The following phases and associated tools were used throughout the term of the assessment:

- Research into the architecture, purpose, and use of the platform.
- Manual code review and walkthrough.
- Manual assessment of the critical Move variables and functions in scope to identify any vulnerability classes related to arithmetic or logic.
- Cross-module call controls.
- Logical controls related to the platform architecture.
- Integration testing using the Aptos Framework.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Integrity (I)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Availability (A)	None (A:N) Low (A:L) Medium (A:M) High (A:H) Critical (A:C)	0 0.25 0.5 0.75 1
Deposit (D)	None (D:N) Low (D:L) Medium (D:M) High (D:H) Critical (D:C)	0 0.25 0.5 0.75 1
Yield (Y)	None (Y:N) Low (Y:L) Medium (Y:M) High (Y:H) Critical (Y:C)	0 0.25 0.5 0.75 1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Scope (s)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

FILES AND REPOSITORY

(a) Repository: satay-movement

(b) Assessed Commit ID: 7996f58

(c) Items in scope:

- packages/satay/sources/vault.move
- packages/satay/sources/base_strategy.move
- packages/satay/sources/protocol.move
- packages/satay/sources/asset.move
- packages/satay/sources/hot_asset.move
- packages/satay/sources/hot_coin.move
- packages/satay/sources/satay.move
- packages/satay/sources/constants.move
- packages/router/sources/auth.move
- packages/router/sources/harvest.move
- packages/router/sources/withdraw.move
- packages/router/sources/deposit.move
- packages/strategies/echelon_simple/sources/strategy.move
- packages/strategies/layerbank_simple/sources/strategy.move
- packages/strategies/moveposition_simple/sources/strategy.move
- packages/strategies/moveposition_simple/sources/basic_ticket.move
- packages/blocks/echelon/sources/echelon_block.move
- packages/blocks/layerbank/sources/layerbank_block.move
- packages/blocks/moveposition/sources/moveposition_block.move
- packages/interfaces/echelon/sources/lending.move
- packages/interfaces/echelon/sources/farming.move
- packages/interfaces/echelon/sources/scripts.move
- Canopyxyz/satay-movement/pull/50/commits/531f1b26afb869852993f9cefe255e01d753e396

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

- 40e3953
- 9f8cc40
- 93516db
- bfb0bc5
- 7bfccc1

- a218cda
- 36d6c65
- 5ade5be
- 4a9c847

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW
0	0	0	3

INFORMATIONAL
8

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
RISK OF BYPASSING MAXIMUM LOSS SAFEGUARD DUE TO UNHANDLED STRATEGY LOSSES	LOW	SOLVED - 02/18/2025
INCORRECT NAME AND SYMBOL ASSIGNED TO SHARES ASSET	LOW	SOLVED - 02/18/2025
PRESENCE OF DISTINCT ERRORS ASSOCIATED WITH THE SAME ERROR CODE	LOW	SOLVED - 02/18/2025
RISK OF OVERPAYING OR INABILITY TO WITHDRAW ASSETS FROM AAVE LENDING POOL	INFORMATIONAL	SOLVED - 02/18/2025
RACE CONDITION COULD PREVENT GOVERNANCE FROM APPROVING THE INTENDED ROUTER	INFORMATIONAL	SOLVED - 02/18/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
UNUSED CODE	INFORMATIONAL	SOLVED - 02/18/2025
INEFFICIENT CODE	INFORMATIONAL	SOLVED - 02/18/2025
MISSING EVENT EMISSION	INFORMATIONAL	SOLVED - 02/19/2025
PRESENCE OF TODO COMMENTS IMPLYING UNFINISHED CODE	INFORMATIONAL	SOLVED - 02/18/2025
PRESENCE OF TYPOGRAPHICAL ERRORS	INFORMATIONAL	SOLVED - 02/18/2025
SUBOPTIMAL INLINE DOCUMENTATION	INFORMATIONAL	SOLVED - 02/18/2025

7. FINDINGS & TECH DETAILS

7.1 RISK OF BYPASSING MAXIMUM LOSS SAFEGUARD DUE TO UNHANDLED STRATEGY LOSSES

// LOW

Description

There is a potential risk that the withdrawal process could bypass the accounting of strategy losses due to the absence of a mandatory enforcement for invoking the `apply_strategy_loss` function between the `withdraw` and `apply_strategy_withdrawal` calls. This gap in the current implementation allows for the possibility that losses are not properly recorded, especially if the router is improperly configured or maliciously altered.

As a result, the maximum loss limit could be exceeded, leading to discrepancies in the user's funds and jeopardizing the integrity of both the strategy and the vault by neglecting to account for the strategy's current debt, losses, and the total value of base assets the vault has allocated to strategies.

Code Location

Below is the implementation of the `vault::apply_strategy_loss` function:

```
1023 public fun apply_strategy_loss(
1024     request: &mut WithdrawalRequest, strategy: Object<BaseStrategy
1025 ) {
1026     assert!(amount <= request.remaining, EINVALID_WITHDRAWAL_AMOUN
1027     request.remaining = request.remaining - amount;
1028 
1029     if (!simple_map::contains_key(&request.losses, &strategy)) {
1030         simple_map::add(&mut request.losses, strategy, amount);
1031         return
1032     };
1033 
1034     let loss = simple_map::borrow_mut(&mut request.losses, &strate
1035     *loss = *loss + amount;
1036 }
```

Below is the implementation of the `vault::apply_strategy_withdrawal` function:

```
1011 public fun apply_strategy_withdrawal(
1012     request: &mut WithdrawalRequest, strategy: Object<BaseStrategy
1013 ) {
1014 
```

```
1015        let asset = hot_asset::destroy(hot_asset);
1016        let amount = fungible_asset::amount(&asset);
1017        assert!(amount <= request.remaining, EINVALID_WITHDRAWAL_AMOUN
1018        request.remaining = request.remaining - amount;
1019
1020        add_debt_offset(request, strategy, amount);
1021        fungible_asset::merge(&mut request.withdrawn, asset);
    }
```

Below is a code snippet of the `router::withdraw_fa` function:

```
148 if (asset_amount < amount) {
149     vault::apply_strategy_loss(&mut request, strategy, amount - as
150 };
151
152 vault::apply_strategy_withdrawal(&mut request, strategy, asset);
```

Below is a code snippet of the `router::withdraw_coin` function:

```
85  if (coin_amount < amount) {
86      vault::apply_strategy_loss(&mut request, strategy, amount - co
87  };
88
89  vault::apply_strategy_withdrawal(&mut request, strategy, hot_asset
```

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:M/D:M/Y:N (4.2)

Recommendation

It is recommended to enforce the execution of `apply_strategy_loss` directly within the **Vault** module when necessary to prevent potential issues.

Remediation Comment

SOLVED: The **Canopy team** solved this issue in the specified commit id.

Remediation Hash

https://github.com/Canopyxyz/satay-movement/commit/40e39533738bea4d64ef1eee171cdb4f0eba059e

7.2 INCORRECT NAME AND SYMBOL ASSIGNED TO SHARES ASSET

// LOW

Description

The Canopy protocol defines three core strategies, each invoking the base strategy within its logic. These strategies can only be created by the protocol governance signer, enabling the addition of a strategy to the specified vault.

However, it was found that the shares asset name was incorrectly set during the creation of the base strategy due to a naming conflict. Specifically, another variable with the same name was mistakenly used to store the symbol prefix, which was then passed as the asset's name. As a result, all shares assets were assigned the same name, 'ss', instead of the name specified in the documentation, which states that the name should be the base asset's name prefixed with 'SS '.

Additionally, it was identified that all shares assets are assigned the same symbol as the corresponding base asset, contrary to the documentation, which specifies that the symbol should be the base asset's symbol prefixed with 'ss'.

Code Location

Below is the implementation of the `create_shares_asset` function:

```
994  fun create_shares_asset(strategy_signer: &signer, base_metadata: O
995      let name = utf8(b"SS "); // e.g "SS Circle", "SS Bitcoin"
996      string::append(&mut name, fungible_asset::name(base_metadata))
997
998      // strategy shares symbols are prefixed with "ss" e.g ssBTC, s
999      let name = utf8(b"ss");
1000     let symbol = fungible_asset::symbol(base_metadata);
1001     let decimals = fungible_asset::decimals(base_metadata);
1002
1003     asset::create(
1004         strategy_signer,
1005         name,
1006         symbol,
1007         decimals,
1008         utf8(b"https://satay.finance/images/default.png"),
1009         utf8(b"https://satay.finance")
1010     )
1011 }
```

Proof of Concept

Scenario

The PoC performs the following steps:
1. Initialize the **protocol** module.
2. Instantiate a base asset metadata.
3. Create a strategy by the governance with the provided base asset metadata.
4. Query the shares asset metadata.
5. Ensure that the shares metadata name equals 'ss'.
6. Ensure that the shares metadata and the base metadata share the same symbol.

Implementation

The following is the implementation of the PoC, which has been added to the existing tests in `packages/satay/tests/base_strategy_tests.move`:

```
#[test]
fun test_incorrect_name_and_symbol_in_shares_metadata() {
    let governance = initialize_state();
    let base_metadata = assets_test_utils::create_asset(option::some(b
    let auth_ref = create_strategy(&governance, option::some(base_meta
    let strategy = base_strategy::auth_ref_strategy(&auth_ref);
    let shares_metadata = base_strategy::shares_metadata(strategy);

    assert!(fungible_asset::name(shares_metadata) == utf8(b"ss"), ETES
    assert!(fungible_asset::symbol(shares_metadata) == fungible_asset:
}
```

Execution

- **Command:**

```
pnpm run at --dev packages/satay --filter test_incorrect_name_and_symb
```

- **Result:**

```
~/Doc/p/m/satay-movement @7996f587 !15 ?2 ❯ pnpm run at --dev packages/satay --filter test_incorrect_name_and_symbol_in_shares_metadata

> @satay-protocol/satay-movement@1.0.0 at /Users/cyberj0e/Documents/projects/move/satay-movement
> node scripts/cli-runner.js aptos move test --check-test-code --skip-fetch-latest-git-deps "--dev" "packages/satay" "--filter" "test_in
correct_name_and_symbol_in_shares_metadata"

Executing in /Users/cyberj0e/Documents/projects/move/satay-movement/packages/satay:
$ aptos move test --check-test-code --skip-fetch-latest-git-deps --dev --filter test_incorrect_name_and_symbol_in_shares_metadata

INCLUDING DEPENDENCY AptosFramework
INCLUDING DEPENDENCY AptosStdlib
INCLUDING DEPENDENCY MoveStdlib
BUILDING Satay
Running Move unit tests
[ PASS    ] 0xbabeeee::base_strategy_tests::test_incorrect_name_and_symbol_in_shares_metadata
Test result: OK. Total tests: 1; passed: 1; failed: 0
{
  "Result": "Success"
}
```

Result of `test_incorrect_name_and_symbol_in_shares_metadata`

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (3.4)

Recommendation

It is recommended to:

- Use a different name for the symbol prefix to avoid overwriting the previously set `name`.
- Add the 'ss' prefix to the shares asset's symbol to clearly distinguish it from the base asset.

Below is an example code snippet that demonstrates how to resolve both issues:

```
let symbol = utf8(b"ss");
string::append(&mut symbol, fungible_asset::symbol(base_metadata));
```

Remediation Comment

SOLVED: The **Canopy team** solved this issue in the specified commit id.

Remediation Hash

https://github.com/Canopyxyz/satay-movement/commit/9f8cc40c7518efc29a278fcfacef4e860da41506

7.3 PRESENCE OF DISTINCT ERRORS ASSOCIATED WITH THE SAME ERROR CODE

// LOW

Description

Distinct and thorough error codes are vital for clear troubleshooting and system reliability. They enable quick identification and resolution of issues, prevent confusion, and enhance the overall robustness of the protocol.

However, in the Satay::vault module, the errors `E_INVARIANT_STRATEGY_INSUFFICIENT_DEBT` and `EINVALID_HOT_ASSET_OWNER` are using the same error code.

Code Location

Below is a code snippet from the **vault** module:

```
291 /// Invariant violation for strategy insufficient debt
292 const E_INVARIANT_STRATEGY_INSUFFICIENT_DEBT: u64 = 133;
293 /// Invalid hot asset owner
294 const EINVALID_HOT_ASSET_OWNER: u64 = 133;
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

It is recommended to assign a distinct error code to each issue to prevent confusion.

Remediation Comment

SOLVED: The **Canopy team** solved this issue in the specified commit id.

Remediation Hash

https://github.com/Canopyxyz/satay-movement/commit/93516db534083fa9811f9c43d3059952a852eaf5

7.4 RISK OF OVERPAYING OR INABILITY TO WITHDRAW ASSETS FROM AAVE LENDING POOL

// INFORMATIONAL

Description

It has been observed that the `withdraw` function within the LayerBank block contains a flaw in its implementation, defined as follows:
1. The signer's aTokens are burned, and the underlying asset is transferred to the recipient's primary fungible store.
2. The asset must then be returned by being extracted from the signer's primary fungible store.
As a result, if the recipient is different than the signer's address, this will lead to two potential outcomes:

- **If the signer has the specified amount of the asset**: The funds will be withdrawn from their balance, which contradicts the intended logic. As a result, the signer ends up burning their aTokens and incurring additional costs through their underlying assets, rather than the recipient incurring the costs. This leads to the recipient receiving double the amount of the asset.
- **If the signer does not have the specified amount of the asset**: The transaction will fail, leading to the signer's underlying asset stuck in aave lending pool.

Note: As of now, this function is only invoked within the LayerBank strategy, with the signer set as the strategy's signer and the recipient set to the signer's address, which should not cause any issues.

Code Location

Below is a code snippet of the `withdraw` function:

```
104 supply_logic::withdraw(account, asset_address, amount as u256, rec
105
106 // Withdraw the asset from the primary fungible store and wrap
107 let metadata = object::address_to_object<Metadata>(asset_address);
108 let asset = primary_fungible_store::withdraw(account, metadata, am
```

Below is the implementation of the `market_withdraw_fa_internal` function:

```
367 fun market_withdraw_fa_internal(strategy_ref: &LayerBankStrategy,
368     let strategy_signer = base_strategy::get_signer(&strategy_ref.
369     let strategy_address = signer::address_of(&strategy_signer);
370
371     // we withdraw the requested amount of the base asset from lay
372         layerbank_block::withdraw(
373         &strategy_signer, // since the supplied assets are registe
```

```
374             object::object_address(&vault::base_metadata(strategy_ref.
375             amount,
376             strategy_address // the recipient of the withdrawn assets
377         )
378     }
```

BVSS

AO:A/AC:L/AX:H/R:P/S:U/C:N/A:M/I:N/D:N/Y:C (1.9)

Recommendation

It is recommended to remove the recipient parameter and manually send it as the signer's address to prevent potential issues when integrating the LayerBank block into future strategies.

Remediation Comment

SOLVED: The **Canopy team** solved this issue in the specified commit id.

Remediation Hash

https://github.com/Canopyxyz/satay-movement/commit/bfb0bc5db63b55d97133c390d9a6e59acf9a34f8

7.5 RACE CONDITION COULD PREVENT GOVERNANCE FROM APPROVING THE INTENDED ROUTER

// INFORMATIONAL

Description

The `request_router_ref` function from the **protocol** module allows any module to set its own address as the `pending_router` address. To validate this address as the router, the governance account must execute the `approve_pending_router` function.
However, this design exposes a vulnerability where protocol griefers can front-run the legitimate governance call by overwriting the `pending_router` value via `request_router_ref` before approval occurs. As a result, the intended router will no longer be eligible for approval, preventing the correct router from being approved.

Code Location

Below is the implementation of the `request_router_ref` function:

```
180 public fun request_router_ref(router_signer: &signer): RouterRef {
181     let router = signer::address_of(router_signer);
182     let protocol_mut = borrow_global_mut<Protocol>(get_address());
183     option::swap_or_fill(&mut protocol_mut.pending_router, router);
184
185     RouterRef { router }
186 }
```

Below is the implementation of the `approve_pending_router` function:

```
159 public entry fun approve_pending_router(account: &signer, router: address)
160     assert!(is_governance(signer::address_of(account)), ENOT_GOVERNANCE);
161
162     let protocol_mut = borrow_global_mut<Protocol>(get_address());
163     assert!(option::is_some(&protocol_mut.pending_router), ENO_PENDING_ROUTER);
164
165     let pending_router = option::extract(&mut protocol_mut.pending_router);
166     assert!(pending_router == router, EINVALID_PENDING_ROUTER);
167     option::swap_or_fill(&mut protocol_mut.router, pending_router);
168 }
```

Proof of Concept

Scenario

The PoC performs the following steps:

1. Initialize the protocol module.
2. The first router requests a router reference from the protocol.
3. The second router requests a router reference from the protocol.
4. The governance account attempts to approve the first pending router by providing its address.
5. The test fails with an `EINVALID_PENDING_ROUTER` error.

Implementation

The following is the implementation of the PoC, which has been added to the existing tests in `packages/satay/tests/protocol_tests.move`:

```
#[test(first_router = @0xDeEF, second_router = @0xCaFe)]
#[expected_failure(abort_code = satay::protocol::EINVALID_PENDING_ROUTI
fun test_race_condition_in_pending_router_requests(first_router: addres
    let satay = initialize();

    // first router request, pending_router will be set as first_route
    let first_router_signer = account::create_account_for_test(first_r
    let first_router_ref = protocol::request_router_ref(&first_router_

    // second router request, pending_router will be set as second_rou
    let second_router_signer = account::create_account_for_test(second
    let _second_router_ref = protocol::request_router_ref(&second_rout

    assert!(!protocol::is_valid_router_ref(&first_router_ref), ETEST_FA

    // approving first_router is impossible as it's no longer the pend
    let router_address = signer::address_of(&first_router_signer);
    protocol::approve_pending_router(&satay, router_address);
}
```

Execution

- **Command:**

```
pnpm run at --dev packages/satay --filter test_race_condition_in_pendi
```

- **Result:**

```
~/Doc/projects/move/satay-movement @7996f587 !16 ?2 ❯ pnpm run at --dev packages/satay --filter test_race_condition_in_pending_router_re
quests

> @satay-protocol/satay-movement@1.0.0 at /Users/cyberj0e/Documents/projects/move/satay-movement
> node scripts/cli-runner.js aptos move test --check-test-code --skip-fetch-latest-git-deps "--dev" "packages/satay" "--filter" "test_ra
ce_condition_in_pending_router_requests"

Executing in /Users/cyberj0e/Documents/projects/move/satay-movement/packages/satay:
$ aptos move test --check-test-code --skip-fetch-latest-git-deps --dev --filter test_race_condition_in_pending_router_requests

INCLUDING DEPENDENCY AptosFramework
INCLUDING DEPENDENCY AptosStdlib
INCLUDING DEPENDENCY MoveStdlib
BUILDING Satay
Running Move unit tests
[ PASS    ] 0xbabeeee::protocol_tests::test_race_condition_in_pending_router_requests
Test result: OK. Total tests: 1; passed: 1; failed: 0
{
  "Result": "Success"
}
```

Result of `test_race_condition_in_pending_router_requests`

BVSS

AO:A/AC:H/AX:M/R:N/S:U/C:N/A:H/I:N/D:N/Y:N (1.7)

Recommendation

It is recommended to change the `pending_router` type to store all pending routers, which will help prevent race conditions. Once a router is approved, it should be removed from the list of pending routers.

Remediation Comment

SOLVED: The **Canopy team** solved this issue by adding a timelock to the update of a pending router in the `protocol::request_router_ref` function. This ensures that governance has a defined, uninterrupted window to approve the router.

The issue was addressed in the following commit IDs:

- 42a745183f98f68f89c8335f0abb9afca5a213a0
- 7bfccc13e354a835a1c1abbab210589a3c6713e9

Remediation Hash

https://github.com/Canopyxyz/satay-movement/commit/7bfccc13e354a835a1c1abbab210589a3c6713e9

7.6 UNUSED CODE

// INFORMATIONAL

Description

Having unused code in the codebase is not a good practice as it increases complexity, can lead to confusion, and may introduce security risks if overlooked.

However, it was found that the `is_signer_authorized` function in the **vault** module, which is a private function, is unused in the module itself.

Code Location

Below is the implementation of the `is_signer_authorized` function:

```
1615 fun is_signer_authorized(account: &signer, vault: Object<Vault>):
1616     let vault_ref = borrow_vault(vault);
1617     let account_address = signer::address_of(account);
1618
1619     let is_manager = vault_ref.manager == account_address;
1620     let is_protocol = protocol::is_governance(account_address);
1621     let is_self = account_address == object::object_address(&vault
1622
1623     is_manager || is_protocol || is_self
1624 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to remove the unused function.

Remediation Comment

SOLVED: The **Canopy team** solved this issue by utilizing the `is_signer_authorized` function within the `create_debt_internal` function.

Remediation Hash

https://github.com/Canopyxyz/satay-movement/commit/a218cdaa2bb46368e72beffc1d6001e7b2988394

7.7 INEFFICIENT CODE

// INFORMATIONAL

Description

In the `tend_internal` function of the **base_strategy** module, it was observed that when updating the `total_idle` and `total_debt` properties, the value of the `total_idle` property is assigned to the `amount` variable. Then, the `total_idle` property is updated by subtracting `amount`, which always results in zero, making this calculation redundant.

Additionally, the `_packet` parameter is unused in both the `deposit::deposit_fa` and `withdraw::withdraw_fa` functions.

Code Location

Below is a code snippet of the `tend_internal` function:

```
976 | // we tend the entire idle balance
977 | let amount = strategy_mut.total_idle;
978 | strategy_mut.total_idle = strategy_mut.total_idle - amount;
979 | strategy_mut.total_debt = strategy_mut.total_debt + amount;
```

Below is a code snippet of the `deposit::deposit_fa` function:

```
40 | public(friend) fun deposit_fa(
41 |     vault: Object<Vault>,
42 |     strategy: Object<BaseStrategy>,
43 |     _packet: Option<vector<u8>>,
44 |     amount: u64
45 | ) {
46 |     let borrowed_router_ref = auth::borrow_router_ref();
47 |     let vault_signer = vault::get_signer_for_router(vault, auth::b
48 |     auth::return_router_ref(borrowed_router_ref);
49 |
50 |     let concrete_address = base_strategy::concrete_address(strateg
51 |     if (concrete_address == @echelon_simple) {
52 |         echelon_simple::vault_deposit_fa(&vault_signer, strategy,
53 |     } else if (concrete_address ==@layerbank_simple) {
54 |         layerbank_simple::vault_deposit_fa(&vault_signer, strategy
55 |     };
56 | }
```

Below is a code snippet of the `withdraw::withdraw_fa` function:

```
46  public(friend) fun withdraw_fa(
47      account: &signer,
48      request: &mut WithdrawalRequest,
49      strategy: Object<BaseStrategy>,
50      _packet: Option<vector<u8>>,
51      amount: u64,
52      max_loss: Option<u64>
53  ): HotAsset {
54      let concrete_address = base_strategy::concrete_address(strateg
55      if (concrete_address == @echelon_simple) {
56          return echelon_simple::vault_withdraw_fa(account, request,
57      } else if (concrete_address == @layerbank_simple) {
58          return layerbank_simple::vault_withdraw_fa(account, reques
59      };
60
61      abort EUNKNOWN_STRATEGY
62  }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to set `strategy_mut.total_idle` to zero directly, eliminating the need for unnecessary computation.

```
let amount = strategy_mut.total_idle;
strategy_mut.total_idle = 0;
strategy_mut.total_debt = strategy_mut.total_debt + amount;
```

Additionally, remove the unused `_packet` parameter from the aforementioned functions.

Remediation Comment

SOLVED: The **Canopy team** solved this issue by applying the recommended change in the `tend_internal` function. As for the unused `packet` parameter, the team stated that it is now used when depositing and withdrawing fungible assets to and from **MovePosition** using virtual coin types.

Remediation Hash

https://github.com/Canopyxyz/satay-movement/commit/36d6c650b9ba790bd7142d07ede4aa8325214a35

7.8 MISSING EVENT EMISSION

// INFORMATIONAL

Description

The **Satay** package implements governance-only functions that modify critical state variables without emitting events. The affected functions are the following:

- `set_protocol_fee`
- `set_performance_fee`
- `set_management_fee`
- `approve_pending_router`

The absence of events prevents external systems or users from tracking critical administrative changes via event logs. This lack of transparency diminishes visibility into owner actions that impact the protocol's behavior.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to emit events within the aforementioned functions.

Remediation Comment

SOLVED: The **Canopy team** solved this issue in the following commit IDs:

- 365e24148f3df07cc5ae94c2559db6a7aec17c85
- 5ade5bedf84bfd3ac2afbb8986fc77d94f8a01c0

Remediation Hash

https://github.com/Canopyxyz/satay-movement/commit/5ade5bedf84bfd3ac2afbb8986fc77d94f8a01c0

7.9 PRESENCE OF TODO COMMENTS IMPLYING UNFINISHED CODE

// INFORMATIONAL

Description

Open TODOs can point to architecture or programming issues that still need to be resolved.

Code Location

Below are the identified TODO comments:

```
packages/strategies/layerbank_simple/sources/strategy.move
71:    // TODO: investigate how to restrict access to these functions or
improve implementation to consider latest value
93:    // TODO: consider removing this redundant check as withdrawing from the
PFS more than the balance will abort
250:   // TODO: investigate if base assets accidentally transferred directly to
strategy can be tended(i.e. included in idle deposits)

packages/satay/sources/vault.move
1339:  // TODO: put this in a function (or is there a better way to get the
paired coin type as string?)

packages/blocks/layerbank/sources/layerbank_block.move
103:   // TODO: maybe user other intermediary storage for the asset
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to resolve the open TODOs.

Remediation Comment

SOLVED: The **Canopy team** solved this issue in the following commit IDs:

- b75d8cd4abfc745b284fb92a1551a908a5a2c40f
- 7bfccc13e354a835a1c1abbab210589a3c6713e9

Remediation Hash

https://github.com/Canopyxyz/satay-movement/commit/7bfccc13e354a835a1c1abbab210589a3c6713e9

7.10 PRESENCE OF TYPOGRAPHICAL ERRORS

// INFORMATIONAL

Description

Several typographical errors were identified throughout the codebase. Although they do not present direct security risks, addressing them is considered a best practice to ensure clarity and consistency.

Code Location

Below are the identified typographical errors:

```
packages/strategies/layerbank_simple/sources/strategy.move
252:    // rewards in the base asset are autocompounded back in

packages/blocks/moveposition/sources/moveposition_block.move
86:    // we can't juse pass `CoinType` because portfolios only hold notes and
not raw coins

packages/satay/sources/vault.move
335:    // If the deposit limit is set, it must be greater than 0 becuase
setting it to 0 will
617:        remaining: reamining_amount
715:    // We cannot convert the asset to a coin directly here because aptos
does't allow it.
937:    let stratgies = vault_strategies(vault);
1439:    // vault shares symbols are prexed with "sv" e.g svBTC, svUSDT

packages/strategies/moveposition_simple/sources/strategy.move
262:        // so we need to withraw from our deposits in the the yield source

packages/strategies/echelon_simple/sources/strategy.move
397:    // so we need to withraw from our deposits in the the yield source
416:    // so we need to withraw from our deposits in the the yield source

packages/satay/sources/base_strategy.move
424:    // We cannot convert the asset to a coin directly here because aptos
does't allow it.
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to correct the aforementioned typographical errors as follows:

```
autocompounded -> auto-compounded
juse -> just
becuase -> because
reamining -> remaining
does't -> doesn't
stratgies -> strategies
prexed -> prefixed
withraw -> withdraw
```

Remediation Comment

SOLVED: The **Canopy team** solved this issue in the specified commit id.

Remediation Hash

https://github.com/Canopyxyz/satay-movement/commit/4a9c847007b1f4a77ae673581390b746ed5d965e

7.11 SUBOPTIMAL INLINE DOCUMENTATION

// INFORMATIONAL

Description

Inline documentation is essential for ensuring code readability, maintainability, and ease of understanding for current and future developers working on the project.
Although the codebase includes inline documentation, a few issues were identified within it.

Code Location

The following parameters were undocumented:

```
packages/satay/sources/vault.move
328:         total_debt_limit: Option<u64>,
812:         debt_limit: u64

packages/strategies/moveposition_simple/sources/strategy.move
88:         data: vector<u8>,
113:         packet: vector<u8>,
141:         data: vector<u8>,
```

The following parameters were documented, despite not being present:

```
packages/satay/sources/vault.move
316:     /// @param protocol_fee The fee charged by the protocol for each
deposit.

packages/strategies/moveposition_simple/sources/strategy.move
61:     /// @param market The market to be used by the strategy.
```

The following parameter names do not align with the actual function parameters:

```
packages/satay/sources/vault.move
484:     /// @param fee The new duration.
504:     /// @param asset The asset to be deposited.
527:     /// @param shares_asset The shares asset to be withdrawn.
```

The following comment references a `strategy_signer` parameter, which does not exist:

```
packages/satay/sources/vault.move
978:     /// Withdraws shares from a strategy specified by the `strategy_signer`
```

```
parameter.
```

The following comments mention incorrect maximum thresholds for the specified fees. The maximum value for the performance fee should be 2000, while the management fee is capped at 300:

```
packages/satay/sources/vault.move
447:    /// @reverts EINVALID_FEE if the performance fee is not between 0 and
5000.
467:    /// @reverts EINVALID_FEE if the management fee is not between 0 and
5000.
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to address the aforementioned issues in the inline documentation to ensure consistency.

Remediation Comment

SOLVED: The **Canopy team** solved this issue in the specified commit id.

Remediation Hash

https://github.com/Canopyxyz/satay-movement/commit/7bfccc13e354a835a1c1abbab210589a3c6713e9
